

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Sergiu Gherasim, FAF-242

Report

Laboratory Work No. 1

Operating Systems

Topic: The OS as a Resource Manager

First Contact with the Operating System

Checked by:
Patricia Reiţman, *university assistant*

Chişinău – 2026

CONTENTS

Introduction **3**

1 Files and Directories **4**

1.1 Navigating and Inspecting 4

1.2 Creating, Copying, Renaming and Removing 6

1.3 Ownership and Permissions 7

2 Processes **8**

2.1 Listing Processes 8

2.2 A Background Process 10

2.3 The /proc Pseudo-Filesystem 11

3 Memory **13**

4 Devices and Storage **14**

5 Finished Early **16**

Conclusions **18**

INTRODUCTION

In the lecture the operating system was presented as the manager that stands between programs and the hardware. The purpose of this laboratory work is to watch that manager at work from the terminal of a Linux virtual machine, through its four core jobs: managing files, processes, memory and devices. For each of them I ran the commands from the task, looked at the output and wrote down what it shows.

I used a Debian 13 virtual machine with 6 CPU cores and 4 GB of RAM, running in UTM on my laptop. First I created a working folder and asked the OS who I am, which kernel manages the machine and how long it has been running.

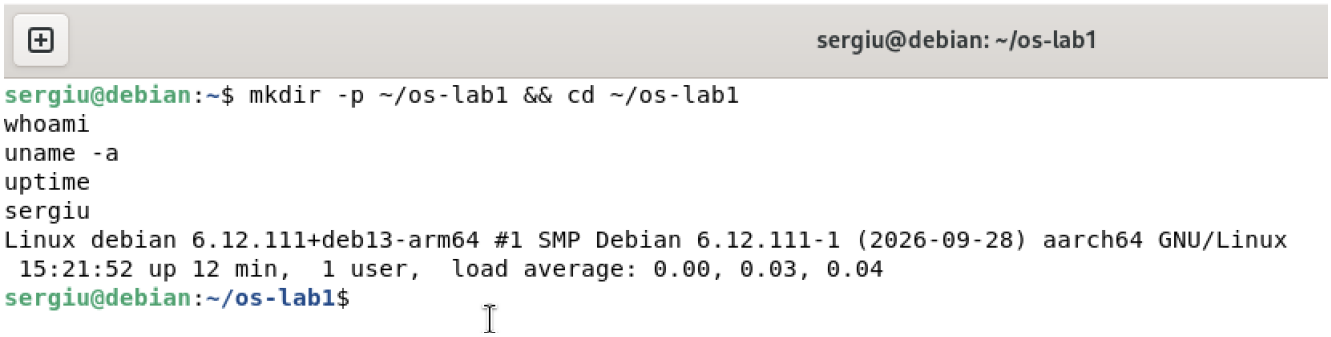


Figure 1: Working folder, whoami, uname -a and uptime

`whoami` prints my user name, `sergiu`. `uname -a` shows that the machine is managed by the Linux kernel 6.12 (`6.12.111+deb13-arm64`), built for the `aarch64` architecture (64-bit ARM). `uptime` shows that the system had been running for 12 minutes with 1 user logged in, and load averages of almost 0 for the last 1, 5 and 15 minutes: the VM was idle.

1 FILES AND DIRECTORIES

1.1 Navigating and Inspecting

```
sergiu@debian:~/os-lab1$ pwd
ls -la /
/home/sergiu/os-lab1
total 76
drwxr-xr-x  19 root root  4096 Oct 12 05:22 .
drwxr-xr-x  19 root root  4096 Oct 12 05:22 ..
lrwxrwxrwx   1 root root     7 Oct  4 13:58 bin -> usr/bin
drwxr-xr-x   4 root root  4096 Oct  4 14:03 boot
drwx------   2 root root  4096 Oct  4 14:02 .cache
drwxr-xr-x  18 root root  3220 Oct 26 15:09 dev
drwxr-xr-x 128 root root 12288 Oct 26 15:09 etc
drwxr-xr-x   3 root root  4096 Oct  4 14:03 home
lrwxrwxrwx   1 root root     7 Oct  4 13:58 lib -> usr/lib
drwx------   2 root root 16384 Oct  4 13:58 lost+found
drwxr-xr-x   3 root root  4096 Oct  4 13:58 media
drwxr-xr-x   2 root root  4096 Oct  4 13:58 mnt
drwxr-xr-x   2 root root  4096 Oct  4 13:58 opt
dr-xr-xr-x 219 root root     0 Oct 26 15:09 proc
drwx------   4 root root  4096 Oct  4 14:11 root
drwxr-xr-x  30 root root   760 Oct 26 15:10 run
lrwxrwxrwx   1 root root     8 Oct  4 13:58 sbin -> usr/sbin
drwxr-xr-x   2 root root  4096 Oct  4 13:58 srv
dr-xr-xr-x  13 root root     0 Oct 26 15:09 sys
drwxrwxrwt  16 root root   400 Oct 26 15:15 tmp
drwxr-xr-x  11 root root  4096 Oct  4 13:58 usr
drwxr-xr-x  11 root root  4096 Oct  4 14:06 var
sergiu@debian:~/os-lab1$
```

Figure 2: pwd and ls -la /

`pwd` prints the directory I am in. `ls -la /` lists the root of the whole tree. Every line starts with the type and permissions, followed by the number of links, the owner, the group, the size, the modification date and the name. Everything under `/` belongs to `root`. `/proc` and `/sys` have the size 0, because they are not on the disk but generated by the kernel, and `/tmp` has the permissions `drwxrwxrwt`, so everybody may create files there. On Debian, `/bin`, `/lib` and `/sbin` are symbolic links (type `l`) into `/usr`.

```
sergiu@debian:~/os-lab1$ ls -la ~
total 116
drwx------ 19 sergiu sergiu  4096 Oct 26 15:21 .
drwxr-xr-x  3 root   root    4096 Oct  4 14:03 ..
-rw-------  1 sergiu sergiu  2536 Oct 26 14:44 .bash_history
-rw-r--r--  1 sergiu sergiu   220 Oct  4 14:03 .bash_logout
-rw-r--r--  1 sergiu sergiu  3526 Oct  4 14:03 .bashrc
drwx------ 10 sergiu sergiu  4096 Oct  4 14:11 .cache
drwx------ 11 sergiu sergiu  4096 Oct 26 15:15 .config
drwxr-xr-x  2 sergiu sergiu  4096 Oct  4 14:07 Desktop
drwxr-xr-x  2 sergiu sergiu  4096 Oct  4 14:07 Documents
drwxr-xr-x  2 sergiu sergiu  4096 Oct  4 14:07 Downloads
-rw-r--r--  1 sergiu sergiu  5290 Oct  4 14:03 .face
lrwxrwxrwx  1 sergiu sergiu     5 Oct  4 14:03 .face.icon -> .face
drwx------  2 sergiu sergiu  4096 Oct 26 14:50 .gnupg
drwx------  4 sergiu sergiu  4096 Oct  4 14:07 .local
drwxr-xr-x  2 sergiu sergiu  4096 Oct  4 14:07 Music
-rw-r--r--  1 sergiu sergiu     0 Oct 26 14:51 note.txt
drwxrwxr-x  2 sergiu sergiu  4096 Oct 26 15:21 os-lab1
drwxrwxr-x  4 sergiu sergiu  4096 Oct 12 06:12 os-lab3
drwxr-xr-x  2 sergiu sergiu  4096 Oct  4 14:07 Pictures
-rw-r--r--  1 sergiu sergiu   807 Oct  4 14:03 .profile
drwxr-xr-x  2 sergiu sergiu  4096 Oct  4 14:07 Public
-rw-rw-r--  1 sergiu sergiu    75 Oct 12 05:57 .selected_editor
drwxrwxr-x  3 sergiu sergiu  4096 Oct 12 07:52 shtest
drwx------  2 sergiu sergiu  4096 Oct  4 14:07 .ssh
-rw-r--r--  1 sergiu sergiu     0 Oct  4 14:12 .sudo_as_admin_successful
drwxr-xr-x  2 sergiu sergiu  4096 Oct  4 14:07 Templates
drwxr-xr-x  2 sergiu sergiu  4096 Oct  4 14:07 Videos
-rw-------  1 sergiu sergiu 10875 Oct 12 08:00 .viminfo
drwxrwxr-x  7 sergiu sergiu  4096 Oct 12 07:49 xv6-riscv
sergiu@debian:~/os-lab1$
```

Figure 3: ls -la ~

The home directory belongs to me and is closed to other users (`drwx---`). Besides my `os-lab1` folder it contains the usual folders of a desktop user (`Desktop`, `Documents`, `Downloads`) and hidden configuration files whose names start with a dot, such as `.bashrc` and `.profile`.

```
sergiu@debian:~/os-lab1$ cd /etc && ls | head
adduser.conf
adjtime
alsa
alternatives
anacrontab
apache2
apparmor
apparmor.d
apt
avahi
sergiu@debian:/etc$
```

Figure 4: cd /etc and ls | head

`/etc` holds the configuration of the whole system: users, groups, network settings and the configuration of installed services.

1.2 Creating, Copying, Renaming and Removing

```
sergiu@debian:/etc$ cd ~/os-lab1
mkdir demo && cd demo
echo "hello operating systems" > note.txt
cp note.txt copy.txt
mv copy.txt renamed.txt
ls -l
rm renamed.txt
total 8
-rw-rw-r-- 1 sergiu sergiu 24 Oct 26 15:23 note.txt
-rw-rw-r-- 1 sergiu sergiu 24 Oct 26 15:23 renamed.txt
sergiu@debian:~/os-lab1/demo$
```

Figure 5: mkdir, echo, cp, mv, ls -l and rm in the demo folder

`mkdir demo && cd demo` creates a directory and enters it. `echo "hello operating systems" > note.txt` creates a file and writes the text into it through output redirection. `cp` makes a second, independent copy of the file, and `mv` renames the copy. There is no separate rename command: inside the same file system `mv` does not copy any data, it only changes the name stored in the

directory. `ls -l` shows both files with the same size of 24 bytes (23 characters and a newline), and `rm` removes the renamed one without printing anything.

1.3 Ownership and Permissions

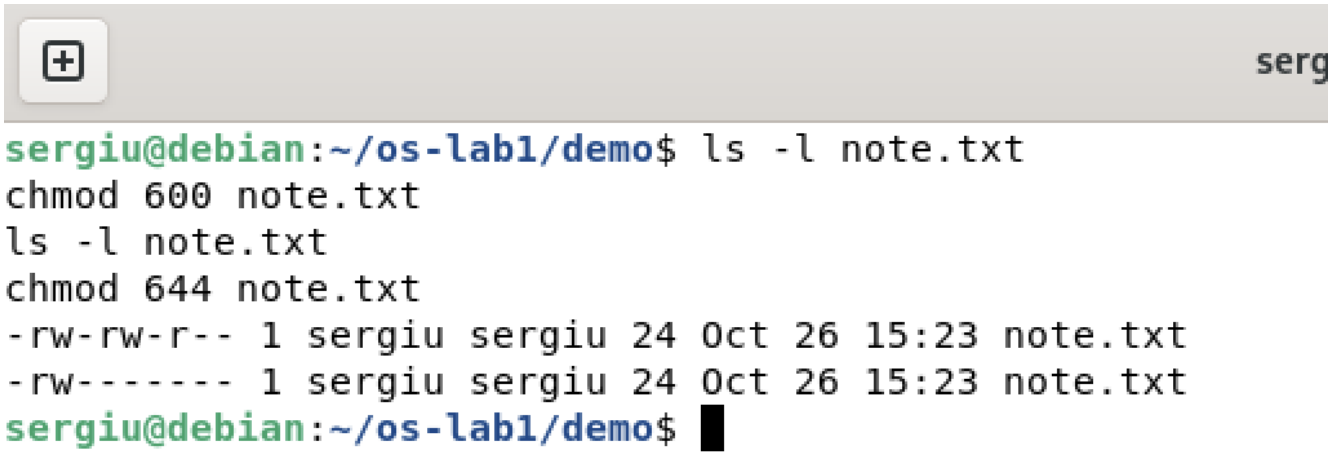


Figure 6: ls -l, chmod 600 and chmod 644 on note.txt

The new file was created as `-rw-rw-r-`: my `umask` is `0002`, so the group also gets write access. Each digit of `chmod` is a sum of $r = 4$, $w = 2$ and $x = 1$, for the owner, the group and others in this order. `chmod 600` leaves read and write only to the owner, and `chmod 644` sets `-rw-r--r-`: read and write for the owner, read only for the group and others.

1. Who owns the files you just created, and which group?

The owner is my user `sergiu` and the group is `sergiu`. On Debian every user gets a private group with the same name.

2. What do the ten characters at the start of `ls -l` mean for note.txt after chmod 600?

After `chmod 600` they are `-rw----`. The first character `-` means a regular file (a directory would have `d`). The next three, `rw-`, mean that the owner can read and write but not execute. The group (`--`) and all other users (`--`) have no rights at all.

3. Name two directories directly under / and what they hold.

`/etc` holds the system-wide configuration files. `/home` holds the home directories of the users, with their personal files.

2 PROCESSES

2.1 Listing Processes

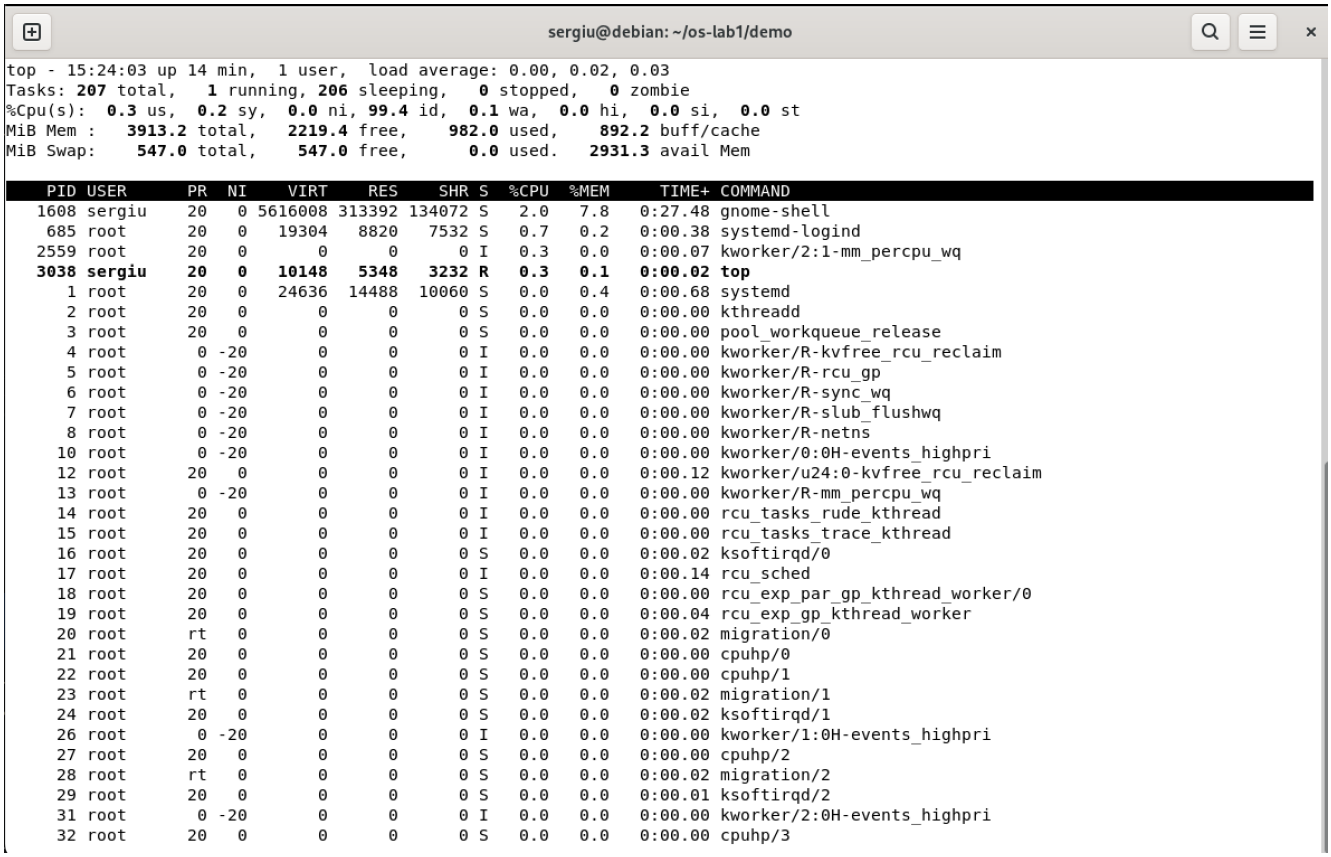


Figure 7: ps aux | head and ps aux | wc -l

`ps aux` lists every process with its owner, PID, CPU and memory usage, state, start time and command. The first line is PID 1. Most of the other processes at the top are kernel threads, whose names are shown in square brackets. `wc -l` counts 210 lines, one of which is the header.

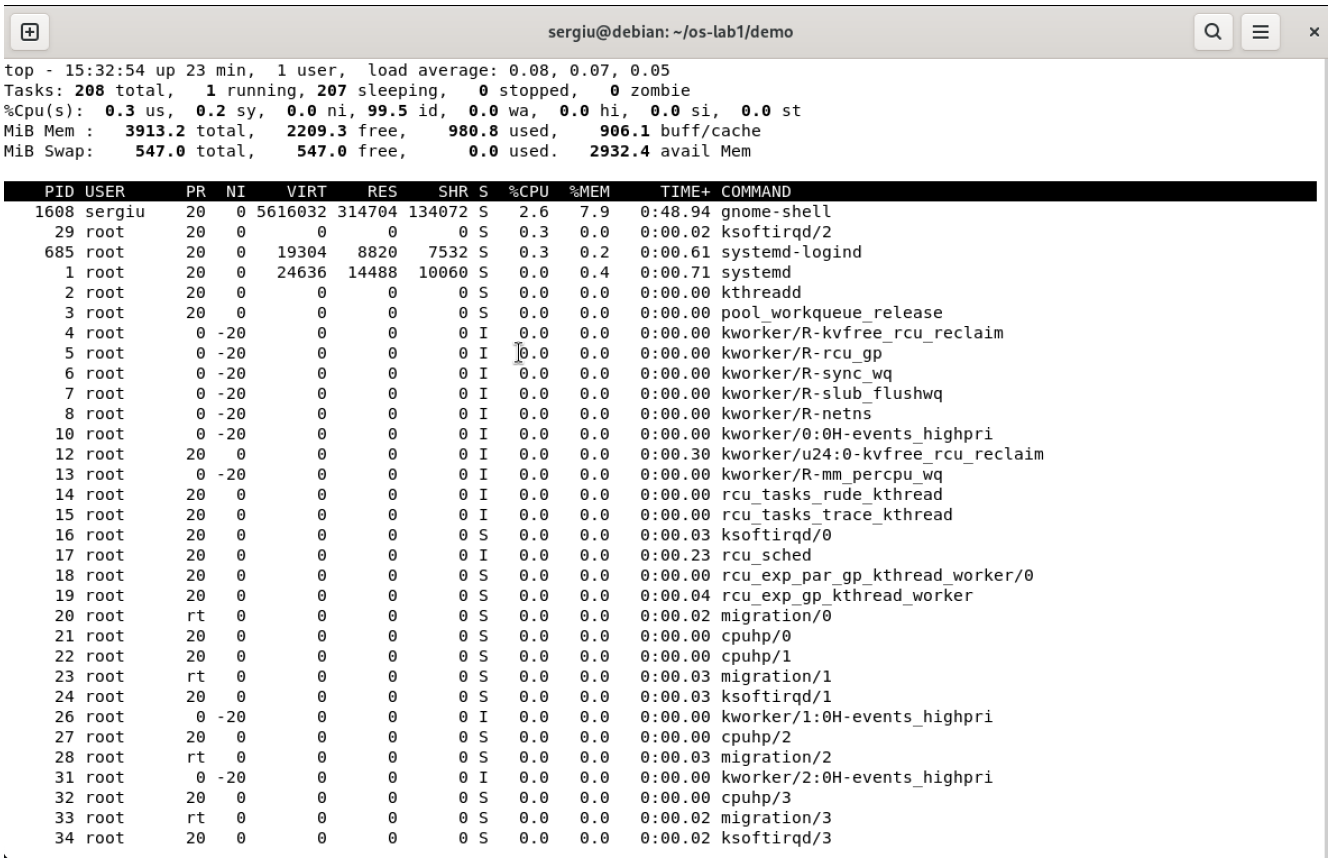


Figure 8: top

`top` shows the same list live, sorted by CPU usage and refreshed every few seconds. Its header summarises the uptime, the load, the number of tasks in each state, the CPU usage and the memory and swap totals. On the idle VM there were 208 tasks, of which only 1 was running (`top` itself) and 207 were sleeping, and the CPU was 99.5 % idle. The busiest process was `gnome-shell`, the desktop, with 2.6 % CPU. Most of the other lines are kernel threads (`kworker`, `ksoftirqd`, `migration`) with no memory of their own; state `I` means an idle kernel thread.

2.2 A Background Process

```
[1] 3073
[1]+  Running                 sleep 300 &
sergiu      3073    2255  0 15:24 pts/0    00:00:00 sleep 300
sergiu      3075    2255  0 15:24 pts/0    00:00:00 grep sleep
bash: kill: (1234) - No such process
bash: sc-10: command not found
[2] 3077
attr        comm             fd                limits     mounts      oom_score      sched         stack    timens_offsets
autogroup   coredump_filter  fdinfo            loginuid   mountstats  oom_score_adj  schedstat     stat     timers
auxv        cpuset           gid_map           map_files  net         pagemap        sessionid     statm    timerslack_ns
cgroup      cwd              io                maps       ns          personality    setgroups     status   uid_map
clear_refs  environ          ksm_merging_pages mem        numa_maps   projid_map     smaps         syscall  wchan
cmdline     exe              ksm_stat          mountinfo  oom_adj     root           smaps_rollup  task
Name:   sleep
Umask:  0002
State:  S (sleeping)
Tgid:   3077
Ngid:   0
Pid:    3077
PPid:   2255
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 24 25 27 29 30 44 46 100 101 102 106 108 1000
NStgid: 3077
NSpid:  3077
NSpgid: 3077
NSsid:  2255
Kthread:        0
VmPeak:     5260 kB
VmSize:     5260 kB
VmLck:         0 kB
bash: syntax error near unexpected token `answers'
    PID COMMAND         COMMAND
      1 systemd         /sbin/init
[2]+  Terminated              sleep 300
lrwxrwxrwx 1 root root 22 Apr 13  2026 /sbin/init -> ../lib/systemd/systemd
sergiu@debian:~/os-lab1/demo$
```

Figure 9: sleep 300 &, jobs, ps -ef | grep sleep and kill

`sleep 300 &` starts `sleep` in the background; the shell prints the job number `[1]` and the PID 3073. `jobs` shows the job as running. `ps -ef | grep sleep` finds it among all processes; its parent PID is the PID of my shell. The second line is the `grep` command itself. `kill` sends the signal `SIGTERM`, and the shell reports the job as terminated.

2.3 The /proc Pseudo-Filesystem

```
sergiu@debian:~/os-lab1/demo$ sleep 300 &
PID=$!
ls /proc/$PID/
cat /proc/$PID/status | head -20
kill $PID
[1] 3503
attr        comm             fd                limits     mounts      oom_score      sched         stack    timens_offsets
autogroup   coredump_filter  fdinfo            loginuid   mountstats  oom_score_adj  schedstat     stat     timers
auxv        cpuset           gid_map           map_files  net         pagemap        sessionid     statm    timerslack_ns
cgroup      cwd              io                maps       ns          personality    setgroups     status   uid_map
clear_refs  environ          ksm_merging_pages mem        numa_maps   projid_map     smaps         syscall  wchan
cmdline     exe              ksm_stat          mountinfo  oom_adj     root           smaps_rollup  task
Name:   sleep
Umask:  0002
State:  S (sleeping)
Tgid:   3503
Ngid:   0
Pid:    3503
PPid:   2255
TracerPid:      0
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
FDSize: 256
Groups: 24 25 27 29 30 44 46 100 101 102 106 108 1000
NStgid: 3503
NSpid:  3503
NSpgid: 3503
NSsid:  2255
Kthread:        0
VmPeak:     5260 kB
VmSize:     5260 kB
VmLck:         0 kB
sergiu@debian:~/os-lab1/demo$
```

Figure 10: ls /proc/$PID/ and cat /proc/$PID/status | head -20

`$!` is the PID of the last background process, so `PID=$!` saves it in a variable. `/proc/$PID` is not stored on any disk: the kernel generates these files when they are read. They describe the process: `cmdline` holds its command line, `fd` its open files, `maps` its memory areas, and `status` a readable summary. In `status` I can see the name `sleep`, the state, the PID 3503 and the parent PID 2255 (my `bash`), my user and group ID 1000, and the virtual memory size of 5260 kB.

```
sergiu@debian:~/os-lab1/demo$ ps -p 1 -o pid,comm,args
ls -l /sbin/init
    PID COMMAND         COMMAND
      1 systemd         /sbin/init
lrwxrwxrwx 1 root root 22 Apr 13  2026 /sbin/init -> ../lib/systemd/systemd
sergiu@debian:~/os-lab1/demo$
```

Figure 11: Process number 1

1. What is the PID of process number 1, and what is it?

PID 1 is `systemd`. It was started as `/sbin/init`, which is a symbolic link to `/lib/systemd/systemd`. It is the first user process, started by the kernel at the end of boot. It starts all the other services, and it becomes the new parent of every process whose parent dies. If it stops, the kernel panics.

2. Roughly how many processes were running on your idle VM?

About 208, according to `top`. `htop` splits them into 109 user processes and 101 kernel threads, so about half are not programs at all but parts of the kernel.

3. What does the "State:" line of /proc/<pid>/status say for your sleeping process?

It says `S (sleeping)`: the process waits for its timer to expire and does not use the CPU until the kernel wakes it up.

3 MEMORY

```
sergiu@debian:~/os-lab1/demo$ free -h
cat /proc/meminfo | head -6
               total        used        free      shared  buff/cache   available
Mem:           3.8Gi       979Mi       2.2Gi        29Mi       892Mi       2.9Gi
Swap:          546Mi          0B       546Mi
MemTotal:        4007168 kB
MemFree:         2274692 kB
MemAvailable:    3003928 kB
Buffers:           36640 kB
Cached:           833184 kB
SwapCached:            0 kB
sergiu@debian:~/os-lab1/demo$
```

Figure 12: free -h and cat /proc/meminfo | head -6

```
sergiu@debian:~/os-lab1/demo$ sleep 300 & PID=$!
grep VmRSS /proc/$PID/status
kill $PID
[2] 3121
VmRSS:      1804 kB
sergiu@debian:~/os-lab1/demo$
```

Figure 13: VmRSS of a sleep process

1. How much total RAM does the VM have, and how much is free?

The VM has 3.8 GiB of RAM (`MemTotal` is 4 007 168 kB). `free -h` shows 979 MiB used, 2.2 GiB free and 2.9 GiB available. The `available` column is more useful than `free`, because it also counts the file cache that the kernel can release when a program needs memory. `free` reads the same numbers that `/proc/meminfo` shows (`MemTotal`, `MemFree`, `MemAvailable`).

2. What is swap, and how much is configured?

Swap is space on the disk where the kernel moves memory pages that are not used often when RAM runs out. This keeps programs running at the cost of speed, because a disk is much slower than RAM. This VM has 546 MiB of swap configured, on its own disk partition `vda4` (see Figure 14), and none of it is used, because more than half of the RAM is still free.

3. How much resident memory (VmRSS) does a bare sleep process use, and does that surprise you?

`VmRSS` is 1804 kB, about 1.8 MiB. At first this surprised me, because `sleep` does nothing. But even the smallest program has its code, a stack and the shared C library mapped into memory. Most of these pages are shared with other processes, so the real cost of one more `sleep` is much smaller.

4 DEVICES AND STORAGE

```
sergiu@debian:~/os-lab1/demo$ df -h
lsblk
Filesystem      Size  Used Avail Use% Mounted on
udev            1.9G     0  1.9G   0% /dev
tmpfs           392M  1.5M  390M   1% /run
/dev/vda3       8.5G  6.5G  1.6G  81% /
tmpfs           2.0G   12K  2.0G   1% /dev/shm
efivarfs        256K   16K  241K   6% /sys/firmware/efi/efivars
tmpfs           1.0M     0  1.0M   0% /run/credentials/systemd-journald.service
tmpfs           5.0M  8.0K  5.0M   1% /run/lock
tmpfs           2.0G   16K  2.0G   1% /tmp
/dev/vda2       760M  9.6M  751M   2% /boot/efi
tmpfs           1.0M     0  1.0M   0% /run/credentials/serial-getty@ttyAMA0.service
tmpfs           392M  100K  392M   1% /run/user/1000
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sr0     11:0    1 1024M  1 rom
vda    254:0    0   10G  0 disk
├─vda1 254:1    0   16M  0 part
├─vda2 254:2    0  760M  0 part /boot/efi
├─vda3 254:3    0  8.7G  0 part /
└─vda4 254:4    0  547M  0 part [SWAP]
sergiu@debian:~/os-lab1/demo$
```

Figure 14: df -h and lsblk

`df -h` lists the mounted file systems with their size and free space. Some of them, such as `tmpfs` and `udev`, are not on a disk at all but in memory. `lsblk` shows the block devices themselves: the 10 GB virtual disk `vda` with four partitions (`vda2` for `/boot/efi`, `vda3` for `/`, `vda4` for swap) and the virtual CD drive `sr0`. The root file system is 8.5 GB and already 81 % full.

```
sergiu@debian:~/os-lab1/demo$ du -sh ~/os-lab1
ls -l /dev | head
12K     /home/sergiu/os-lab1
total 0
crw-r--r--  1 root   root     10, 235 Oct 26 15:09 autofs
drwxr-xr-x  2 root   root         160 Oct 26 15:09 block
drwxr-xr-x  2 root   root          60 Oct 26 15:09 bsg
crw-------  1 root   root     10, 234 Oct 26 15:09 btrfs-control
drwxr-xr-x  3 root   root          60 Oct 26 15:09 bus
lrwxrwxrwx  1 root   root           3 Oct 26 15:09 cdrom -> sr0
drwxr-xr-x  2 root   root        3020 Oct 26 15:10 char
crw-------  1 root   root      5,   1 Oct 26 15:09 console
lrwxrwxrwx  1 root   root          11 Oct 26 15:09 core -> /proc/kcore
sergiu@debian:~/os-lab1/demo$
```

Figure 15: du -sh ~/os-lab1 and ls -l /dev | head

`du -sh` shows that my work folder occupies only 12 KB: three 4 KB blocks, for `os-lab1`, `demo` and `note.txt`. In `ls -l /dev` the first letter is `c` for character devices, which transfer a

stream of bytes (terminals, `null`), or `b` for block devices, which are read in blocks (disks). Instead of a size, a device file shows two numbers (for example `5, 1` for `console`), the major and the minor number, which tell the kernel which driver and which device the file stands for.

```
sergiu@debian:~/os-lab1/demo$ mount | head
sysfs on /sys type sysfs (rw,nosuid,nodev,noexec,relatime)
proc on /proc type proc (rw,nosuid,nodev,noexec,relatime)
udev on /dev type devtmpfs (rw,nosuid,relatime,size=1943976k,nr_inodes=485994,mode=755,inode64)
devpts on /dev/pts type devpts (rw,nosuid,noexec,relatime,gid=5,mode=600,ptmxmode=000)
tmpfs on /run type tmpfs (rw,nosuid,nodev,noexec,relatime,size=400720k,mode=755,inode64)
/dev/vda3 on / type ext4 (rw,relatime,errors=remount-ro)
securityfs on /sys/kernel/security type securityfs (rw,nosuid,nodev,noexec,relatime)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64)
cgroup2 on /sys/fs/cgroup type cgroup2 (rw,nosuid,nodev,noexec,relatime,nsdelegate,memory_recursiveprot)
pstore on /sys/fs/pstore type pstore (rw,nosuid,nodev,noexec,relatime)
sergiu@debian:~/os-lab1/demo$
```

Figure 16: mount | head

`mount` shows that besides the disk, the kernel mounts virtual file systems such as `sysfs` on `/sys`, `proc` on `/proc` and `devtmpfs` on `/dev`.

1. Which device is your root filesystem "/" mounted on?

`/` is mounted on `/dev/vda3`, the third partition of the virtual disk of the VM, formatted as `ext4`.

2. Give one entry from /dev and say what real thing it stands for.

`/dev/vda` is the whole virtual hard disk of the machine. Reading it gives the raw bytes of the disk, below any file system. Another example is `/dev/null`, which stands for no device at all: it discards everything written to it.

3. What does "everything is a file" mean, based on what you saw?

Disks, terminals, processes and kernel information all appear as names in the same file tree (`/dev`, `/proc`) and are used with the same operations as regular files: `open`, `read`, `write` and `close`.

5 FINISHED EARLY

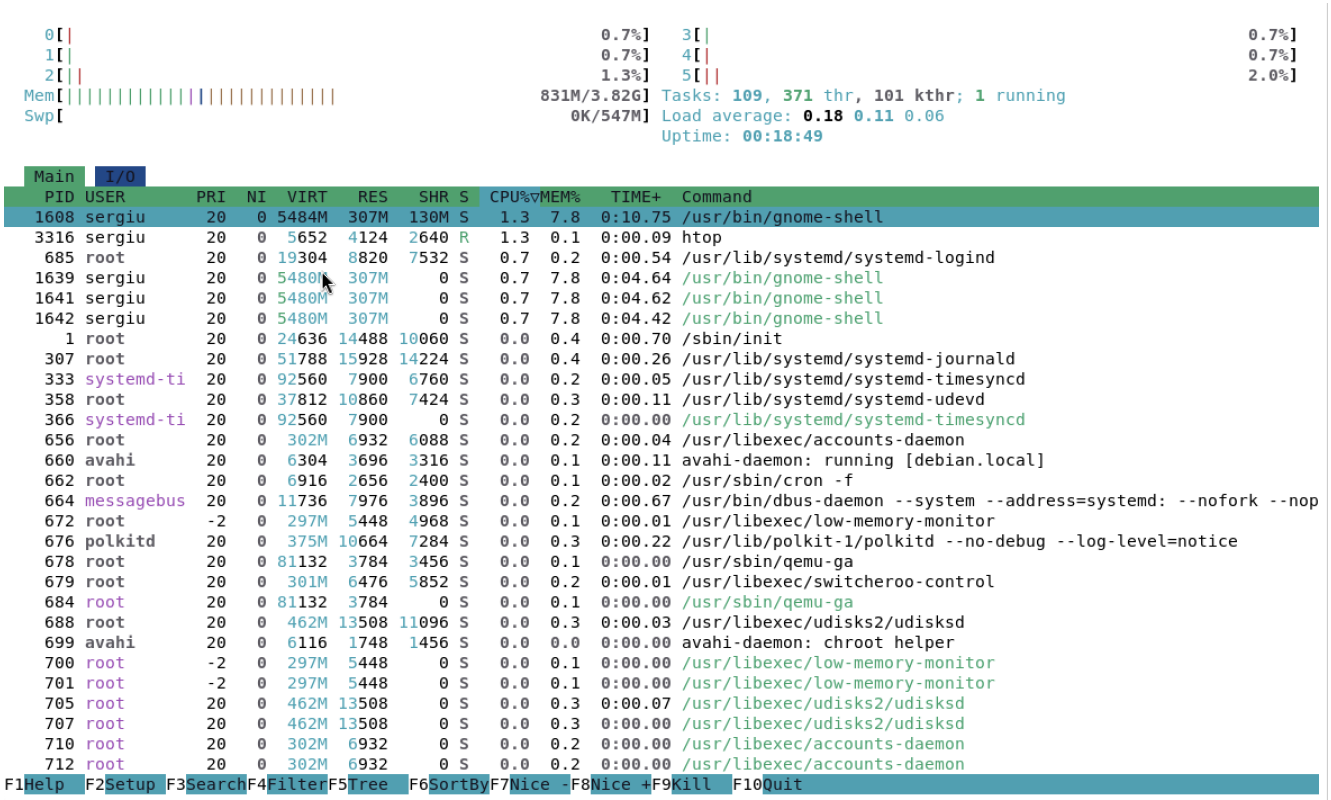


Figure 17: htop

`htop` shows the same information as `top`, but with a bar for every CPU core (6 here) and for memory and swap, and it separates the 109 user processes from the 101 kernel threads. It can also scroll, sort, show processes as a tree and kill them with the keyboard.

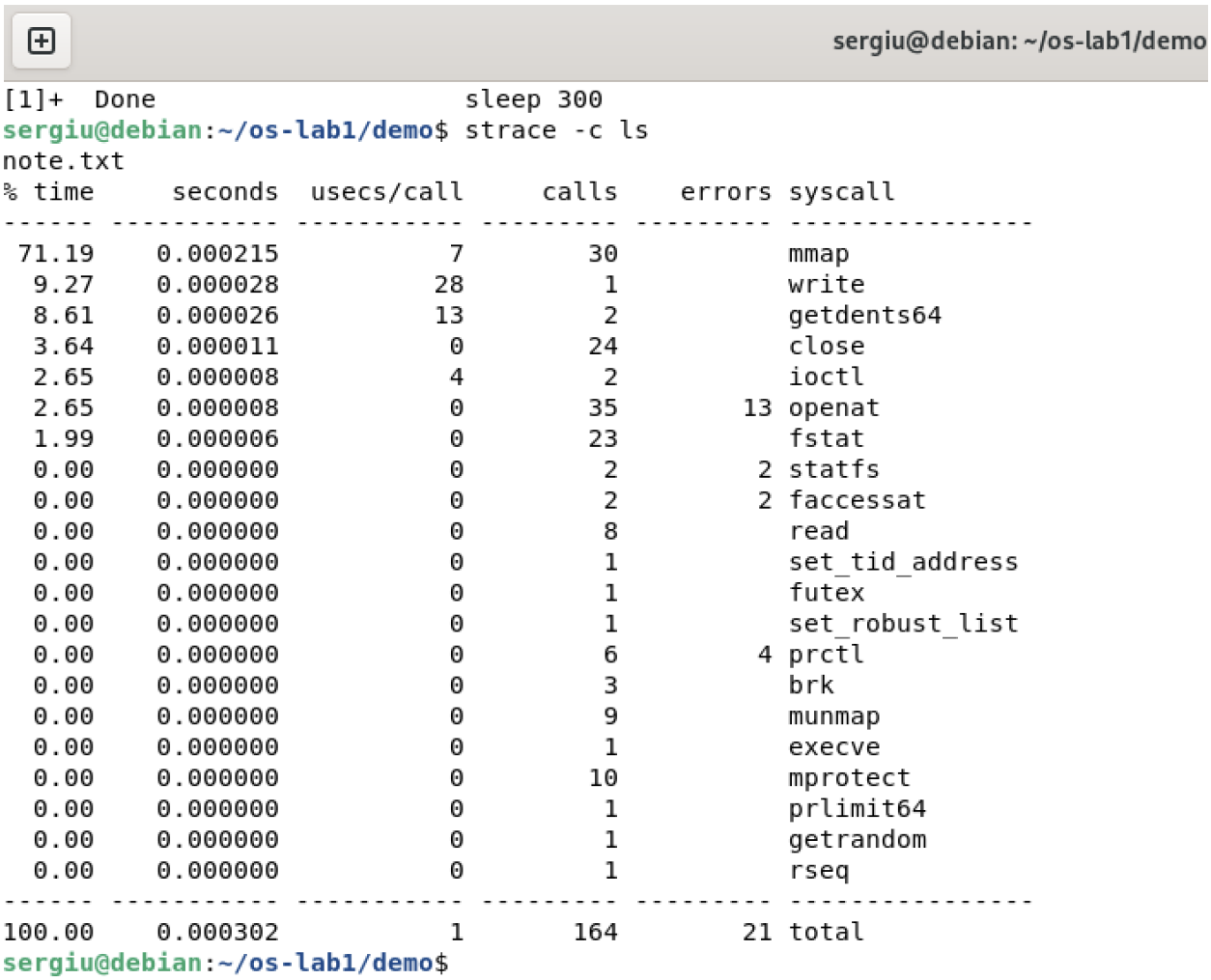


Figure 18: strace -c ls

`strace -c` runs `ls` and counts every system call it makes. Listing a directory with a single file needed 164 calls of 21 different kinds, and 21 of them failed. Most of them only prepare the program: `execve` loads it, then 35 `openat`, 30 `mmap` and 24 `close` calls look for and load the shared libraries and locale files (the 13 failed `openat` calls are files that were looked for but do not exist). The real work is only 3 calls: 2 `getdents64` read the directory entries and 1 `write` prints `note.txt`. Every one of these calls is a request to the OS, because a program cannot touch the disk or the terminal by itself.

CONCLUSIONS

The operating system manages four resources: files, processes, memory and devices. I saw **files** with `ls -l`, which shows the owner and permissions the OS keeps for every name in the tree, and **processes** with `ps aux`, which lists every program the OS has created and is scheduling. I saw **memory** with `free -h`, which shows how the OS shares the physical RAM and swap, and **devices** with `lsblk`, which shows the disks the OS hides behind file-like names in `/dev`.